Secure File Upload System

Project Synopsis

Student: [Student name]

Enrollment number: [Enrollment number]

Programme and semester: [Programme and semester]

Department: [Department]

Institution: [College or university]

Project guide: [Guide name and designation]

Academic year: [Academic year]

Academic project documentation

Contents

Abstract

Problem statement

Aim and objectives

Background and research basis

Existing approach and proposed system

Scope and modules

Interface and interaction design

Controlled sharing design

Functional requirements

Nonfunctional requirements

Technology and resource requirements

Feasibility study

SDLC methodology

System design overview

Testing and evaluation

Expected outcome and limitations

References

Abstract

The Secure File Upload System is a web application that demonstrates how untrusted files can be accepted through a controlled processing pipeline. A user creates an account, signs in and uploads a permitted text or image file. The server validates the filename, declared MIME type, file signature, content structure and resource limits. Images are decoded and re-encoded to remove metadata and appended data. Accepted content is encrypted before being stored outside the public web directory. Downloads require ownership verification and an integrity check.

The proposed system uses Python, Flask, SQLite, Pillow, cryptography, PyOTP and Waitress. Additional controls include cross-site request forgery protection, memory-hard password hashing, optional time-based two-factor authentication, revocable sessions, account quotas, rate limiting and a tamper-evident activity log. Controlled sharing uses short-lived, single-use codes. A local ClamAV adapter supports antivirus scanning when configured; the interface explicitly identifies files that were not scanned. Evaluation combines automated functional and security tests with a browser demonstration. The project follows an iterative secure software development life cycle covering planning, requirements, design, implementation, verification, deployment and maintenance.

Problem statement

An unrestricted upload form can allow executable files, misleading extensions, malformed images, oversized requests and unintended disclosure of private content. Filename and browser-provided MIME checks alone cannot establish that a file is appropriate to store. Images may also expose author, device or location metadata. A practical student project must show both preventive controls and the conditions under which those controls remain effective.

This project addresses that problem by combining independent checks with private encrypted storage and per-user authorization. Its scope is deliberately limited to TXT, PNG, JPG and JPEG so the validation rules can be explained and tested thoroughly.

Aim and objectives

The aim is to implement and evaluate a secure file upload workflow that is understandable, reproducible and suitable for an academic demonstration.

• Authenticate users and restrict each file operation to its owner.

• Validate extension, declared MIME, magic bytes and decoded content rather than trusting a single signal.

• Enforce a 5 MiB file limit, a 16-megapixel image limit, a 50 MiB account quota and a 200-file count limit.

• Strip image metadata by re-encoding supported images and store only the normalized output.

• Encrypt accepted files with authenticated encryption and verify them before download.

• Support optional TOTP two-factor authentication and immediate session revocation.

• Record security-relevant events in a per-account HMAC chain.

• Share selected files through expiring, single-use codes that the owner can revoke.

• Provide repeatable tests, deployment instructions and a documented secure SDLC.

Background and research basis

OWASP's File Upload Cheat Sheet recommends combining allowlists, content inspection, storage separation, access restrictions and resource controls [R1]. The project applies those ideas to a small and explicitly bounded format set. OWASP's password guidance supports memory-hard password hashing; the implementation uses the documented scrypt configuration N=32768, r=8 and p=3 [R2]. NIST SP 800-218 provides the secure-development reference used to organize development and verification activities, without claiming formal compliance [R3].

The cryptography library's Fernet recipe supplies authenticated encryption rather than a custom cryptographic construction [R4]. PyOTP implements TOTP and identifies replay rejection and throttling as necessary integration controls [R5]. The research register explains the selected sources and the limits of each security claim.

Existing approach and proposed system

The comparison below describes a basic educational upload baseline, not a measured assessment of a specific commercial service.

Aspect	Basic upload baseline	Proposed implementation
Type check	Filename extension only	Extension, declared MIME, signature and decoder
Storage	Original filename in public folder	Random ID in private encrypted storage
Privacy	Uploaded metadata retained	Re-encoded images with metadata removed
Access	Direct file URL	Authenticated owner check on every request
Resource control	No explicit quota	Per-file, pixel, account-byte and file-count limits
Accountability	Minimal logs	Per-user activity with integrity verification
Authentication	Password only	Password with optional TOTP and revocable sessions

Scope and modules

The proposed modules are account registration and sign-in, session management, optional two-factor authentication, file validation and normalization, optional antivirus integration, encryption and storage, authorized download and deletion, controlled sharing, audit logging, and the web interface. Deployment assets include a Waitress entry point, container configuration and continuous-integration checks.

The initial release excludes PDF and Office document processing, SVG, archives, executables, unrestricted public links, administrator browsing of user files, password recovery, cloud hosting and machine-learning malware detection. These are separate extensions, not implied capabilities. Metadata stripping applies to supported images, not arbitrary text content. A recipient holding an explicitly issued share code is authorized for that code's permitted download; files otherwise remain private.

Interface and interaction design

The landing page will combine neobrutalism and neoclassical typography: bold outlines, offset shadows and geometric elements paired with serif headings, generous spacing and a restrained editorial composition. The design will be original, using the supplied references for inspiration rather than reproducing their layouts or branding.

The authenticated workspace will prioritize a clear sidebar, searchable file table, upload controls, storage usage, file details and sharing management. Arrow and transfer animations will communicate movement and progress, with reduced-motion support. Progress indicators must reflect actual transfer or processing states rather than fabricated percentages. Keyboard access, labeled forms and readable error messages will remain usable without animation.

Controlled sharing design

An authenticated owner can issue a random 96-bit share code for a selected file. The proposed policy permits one download within 15 minutes and allows the owner to revoke the code sooner. The database stores only a hash of the code. Recipients enter the code through a rate-limited form; the server checks expiry, revocation and usage, verifies file integrity and atomically consumes the code before delivery. Codes are bearer credentials and must be sent through a trusted channel. They are not included in application audit details or ordinary download URLs.

The sharing test plan covers successful redemption, invalid codes, expiry, reuse, revocation, cross-owner management attempts and concurrent redemption. The ability to share deliberately changes the authorization model and must be evaluated independently from owner-only downloads.

Functional requirements

ID	Requirement	Acceptance condition
FR01	Register and authenticate an account	Valid credentials succeed; invalid input is rejected
FR02	Upload allowed content	TXT, PNG and JPEG pass their respective validation rules
FR03	Reject unsafe requests	Invalid type, structure, name or size creates no file record
FR04	Store and retrieve private files	Ciphertext is stored; direct downloads require ownership
FR05	Remove image metadata	Downloaded image excludes test EXIF and trailing markers
FR06	Manage account security	TOTP, logout and all-device revocation behave as documented
FR07	Inspect activity	User sees only their events and an audit-integrity result
FR08	Integrate antivirus	Scanner rejection/error prevents storage; absent scanner is clearly disclosed
FR09	Share a selected file by code	Only a valid, unused, unexpired and unrevoked code permits redemption

Nonfunctional requirements

The interface must support keyboard operation, visible labels and small-screen layouts. Data must persist across normal restarts. SQL statements involving user input must use parameters. Files must remain inaccessible through the static-file route. Security errors must not expose stack traces or private keys. The system targets a single small deployment; performance and concurrency must be measured before widening that scope. Tests must use isolated data and preserve the live vault.

Technology and resource requirements

Area	Selection
Backend	Python 3.12, Flask 3.1.3, Waitress 3.0.2
Frontend	Jinja templates, HTML5, CSS, small JavaScript enhancements
Persistence	SQLite with foreign keys, WAL mode and parameterized queries
File processing	Pillow 12.3.0; explicit PNG and JPEG signatures
Cryptography	cryptography 50.0.1 Fernet, SHA-256, HMAC-SHA-256, Werkzeug scrypt
Authentication	Flask-WTF CSRF, Flask-Limiter, PyOTP 2.10.0, local QR generation
Testing and delivery	pytest, Docker assets, GitHub Actions, Bandit, pip-audit
Suggested development machine	64-bit computer, 4 GB RAM, 1 GB free disk, current browser

The hardware values are planning estimates, not measured minimum requirements. The core local demonstration does not require paid cloud services. Optional antivirus signatures, package installation and hosted automation require network access.

Feasibility study

Technical feasibility is supported by a small Flask application, SQLite and maintained processing libraries. Economic feasibility comes from using open-source components and local execution; hosting and operating costs depend on deployment. Operational feasibility depends on simple upload and download flows and understandable errors. Schedule feasibility is improved by restricting formats and separating optional integrations from the core pipeline. Privacy feasibility requires avoiding sensitive demonstration data and controlling access to the data directory and backups.

SDLC methodology

The project follows an iterative model with security tasks in every phase. A stage is complete when its output can be reviewed or tested, not simply when code is written.

Phase	Activities	Deliverable and exit condition
Planning	Define problem, stakeholders, scope and constraints	Synopsis with agreed objectives and exclusions
Requirements	Capture use cases, misuse cases and acceptance criteria	SRS and requirement traceability

Phase	Activities	Deliverable and exit condition
Analysis and design	Threat model, data flow, schema, trust boundaries	Reviewed architecture, ERD and validation rules
Implementation	Develop modules incrementally and review code	Runnable application and versioned configuration
Verification	Positive, negative, isolation and integrity tests	Test evidence and documented unresolved limitations
Deployment	Configure process, keys, storage, HTTPS and backups	Successful smoke test in the chosen environment
Maintenance	Review dependencies, logs, scanner signatures and recovery	Periodic checks and change-driven regression tests

System design overview

Browser requests enter Flask through Waitress. Authentication and CSRF checks protect relevant routes. Upload processing moves from bounded input to filename/type checks, image normalization or text validation, optional scanning, quota enforcement and encryption. SQLite stores users, sessions, file metadata, share-code hashes and event records. The private filesystem stores encrypted blobs and key material. Direct downloads require ownership; shared downloads require a valid single-use capability. Both paths require successful decryption and a matching SHA-256 digest. The full report will include the architecture, DFD, ERD, use cases and sequence diagram.

Testing and evaluation

The evaluation uses isolated pytest fixtures and browser checks. Tests cover valid uploads, false extensions, MIME mismatch, malformed images, metadata removal, oversized files, authorization bypass attempts, missing CSRF tokens, ciphertext corruption, copied-session replay, TOTP replay and audit modification. Scanner return-code tests are controlled integration tests; a real scanner and current signatures must be separately verified before claiming antivirus operation. Test results belong to the exact tested version and do not prove the absence of all vulnerabilities.

Expected outcome and limitations

The outcome is a locally runnable file-management application and an academic submission package with a synopsis, report, diagrams, tests, source references and demonstration instructions. The project demonstrates multiple security layers and gives the student concrete behaviors to explain.

Key limitations include shared server-side key storage, no account recovery, optional antivirus, an in-memory single-process rate limiter, and the absence of public-deployment or penetration-test certification. The HMAC audit chain is tamper-evident for retained records but cannot prove that a complete final suffix was not deleted without an independent checkpoint. The full security document records these boundaries and proposed improvements.

References

[R1] OWASP. File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[R2] OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[R3] NIST. SP 800-218, Secure Software Development Framework Version 1.1. February 2022. https://csrc.nist.gov/pubs/sp/800/218/final

[R4] Python Cryptographic Authority. Fernet documentation. https://cryptography.io/en/stable/fernet/

[R5] PyOTP. Official documentation. https://pyauth.github.io/pyotp/

Web sources consulted during preparation on 16-17 September 2026. Bibliographic details should be converted to the institution's required reference style if necessary.